

18. Stock Buy and Sell – Max 2 Transactions Allowed

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Solution {
public:
    int maxProfit(vector<int> &prices) {
        int n = prices.size();
        if (n < 2) return 0;
        vector<int> left(n, 0), right(n, 0);
        // left[i] = max profit for [0..i] with at most one transaction
        int minPrice = prices[0];
        for (int i = 1; i < n; ++i) {
            minPrice = min(minPrice, prices[i]);
            left[i] = max(left[i-1], prices[i] - minPrice);
        } // right[i] = max profit for [i..n-1] with at most one transaction
        int maxPrice = prices[n-1];
        for (int i = n-2; i >= 0; --i) {
            maxPrice = max(maxPrice, prices[i]);
            right[i] = max(right[i+1], maxPrice - prices[i]);
        }

        // combine the two
        int ans = 0;
        for (int i = 0; i < n; ++i) {
            ans = max(ans, left[i] + right[i]);
        }
        return ans;
    }
};
```

The screenshot displays a C++ IDE interface. On the left, the 'Output Window' shows 'Problem Solved Successfully' with a green checkmark. Below this, it indicates 'Test Cases Passed: 1120 / 1120', 'Attempts: Correct / Total: 2 / 2', and 'Accuracy: 100%'. The 'Time Taken' is 0.29. A note states: 'You get marks only for the first correct submission if you solve the problem without viewing the full solution.' At the bottom, there are buttons for 'Solve Next' and 'Stickler Thief', 'Maximum Product Subarray', and 'Rod Cutting'.

The right panel shows the C++ code for the solution, which is the same code as provided in the previous blocks. The code is written in C++ and uses a dynamic programming approach to find the maximum profit with at most two transactions.